

Building Firefox On Windows

This document will help you get set up to build Firefox on your own computer. Getting set up can take a while - we need to download a lot of bytes! Even on a fast connection, this can take ten to fifteen minutes of work, spread out over an hour or two.

If you'd prefer to build Firefox for Windows in a virtual machine, you may be interested in the [Windows images provided by Microsoft](#).

! Note

Microsoft “temporarily” stopped providing these VM images in 2024. Although more time-consuming initially, you can create a VM and install Windows with an official Windows 11 ISO file from [Microsoft's Download Windows 11 page](#).

Unlike the evaluation versions in Microsoft's pre-built VM images, an unactivated Windows VM does not shut down every hour.

! Note

The Mozilla Build package is for now only compiled for x64. It will run fine via emulation on Windows on ARM machines (the official name for Windows running on an aarch64 CPU), but the Firefox build system will be (rightfully) confused when being invoked on a Windows on ARM machine by a x64 Python, and won't know what to do.

This will be fixed, but in the meantime, running `./mach` using a native Windows on ARM Python will make everything work appropriately, including bootstrapping the right tool-chains and such.

The Python project provides an [official installer](#) for Windows on ARM.

Invoke it like so:

```
C:\path\to\python.exe ./mach build
```

System Requirements

- **Memory:** 4GB RAM minimum, 8GB+ recommended.
- **Disk Space:** At least 40GB of free disk space.
- **Operating System:** Windows 10 or later. It is advisable to have Windows Update be fully up-to-date. See [Supported Build Hosts](#) for more information.

Recommended (For Windows 11 Users)

Setup a [Dev Drive](#).

Note

- A Dev Drive has been shown to make Firefox builds and VCS operations 5-10% faster.
- This guide assumes no Dev Drive, so all instructions of `C:\mozilla-source` should be to your Dev Drive letter instead (eg: `D:\mozilla-source`), as your `C:\` drive can never be a Dev Drive.
- If you are using a Virtual Machine, assign a second disk in the VM hypervisor and use that disk as the disk partition for the Dev Drive.

1. Install MozillaBuild

Install [MozillaBuild](#).

Accept the default installation directory. Windows may prompt you to “reinstall with the correct settings”, which you should click to accept.

When working with Firefox tooling, you’ll need to do so from within the MozillaBuild shell. You can start it by running `C:\mozilla-build\start-shell.bat` (you may want to make a shortcut to this file so it’s easier to start).

Note

The MozillaBuild shell is a lot more like a Linux shell than the Windows `cmd`. The installer bundles required tools such as Git, Python, wget, etc. You can learn more about it [here](#).

2. Bootstrap a copy of the Firefox source code

Now that your system is ready, we can download the source code and have Firefox automatically download the other dependencies it needs. The below command will download a lot of data (years of Firefox history!) then guide you through the interactive setup process.

```
# Using the C:\mozilla-build\start-shell.bat shell from step 1:
cd c:/
mkdir mozilla-source
cd mozilla-source
wget https://raw.githubusercontent.com/mozilla-
firefox/firefox/refs/heads/main/python/mozboot/bin/bootstrap.py

python3 bootstrap.py
```

Note

When running `bootstrap.py` there will be a [UAC \(User Account Control\) prompt](#) for PowerShell after selecting the destination directory for the source code clone. This is necessary to add the Microsoft Defender Antivirus exclusions automatically. You should select `Yes` on the UAC prompt, otherwise you will need to [follow some manual steps below](#).

Choosing a build type

If you aren't modifying the Firefox backend, then select one of the [Artifact Mode](#) options. If you are building Firefox for Android, you should also see the [GeckoView Contributor Guide](#).

Ensure antivirus exclusions

Microsoft Defender Antivirus and some third-party antivirus products are known to significantly degrade build times and sometimes even cause failed builds (due to a “missing file”). This is usually because we have tests

for well-known security bugs that have code samples that antivirus software identifies as a threat, automatically quarantining/corrupting the files.

To avoid this, add the following folders to your third-party antivirus exclusion list:

- The `C:\mozilla-build` folder.
- The directory where the Firefox code is (probably `C:\mozilla-source`).
- The `%USERPROFILE%\mozbuild` directory (probably `C:\Users\
<user>\.mozbuild`).

The `bootstrap.py` script attempts to add the above folders to the Microsoft Defender Antivirus exclusion list automatically. You should check that they were successfully added, but if they're missing you will need to [add the exclusions to Microsoft Defender Antivirus manually](#).

Note

If you are using Git and you're already missing files (you'll see them listed in `git status`), you can have them brought back by discarding changes in your source tree: `git restore .`.

3. Build

Now that your system is bootstrapped, you should be able to build!

```
cd c:/mozilla-source/firefox
git pull origin main
./mach build
```

 Congratulations! You've built your own home-grown Firefox! You should see the following message in your terminal after a successful build:

```
Your build was successful!
To take your build for a test drive, run: |mach run|
For more information on what to do now, see https://firefox-source-
docs.mozilla.org/setup/contributing_code.html
```

You can now use the `./mach run` command to run your locally built Firefox!

If your build fails, please reference the steps in the [Troubleshooting section](#).

Now the fun starts

Time to start hacking! You should join us on [Matrix](#), say hello in the [Introduction channel](#), and [find a bug to start working on](#). See the [Firefox Contributors' Quick Reference](#) to learn how to test your changes, send patches to Mozilla, update your source code locally, and more.

Note

If you'd like to interact with Mach from a different command line environment than MozillaBuild, there's experimental support for it described [over here](#).

Troubleshooting

Build errors

If you encounter a build error when trying to setup your development environment, please follow these steps:

1. Copy the entire build error to your clipboard
2. Paste this error on gist.github.com in the text area
3. Go to the [introduction channel](#) and ask for help with your build error. Make sure to post the link to the gist.github.com snippet you created!

The CLOBBER file has been updated

This is a normal error to encounter and tends to appear when working on a bug for a long period of time. If you encounter this error, you need to run `./mach clobber` before running `./mach build`. Running `./mach clobber` will remove previous build artifacts to restart a build from scratch. If you are using an artifact build, this will mean that the next build will take slightly longer than usual. However, if you are using a non-artifact/full build, the next build will take significantly longer to complete.

MozillaBuild out-of-date

The build system expects that you're using the most-recent MozillaBuild release. However, MozillaBuild doesn't auto-update. If you're running into local issues, they may be resolved by [upgrading your MozillaBuild](#).

Spaces in folder names

Firefox will not build if the path to MozillaBuild or the Firefox source contain **spaces** or other breaking characters such as pluses, quotation marks, or metacharacters. Windows usernames are a common cause for spaces in the path, so please ensure that your Windows username does not contain spaces, or miniconda will have errors during fenix builds. The Visual Studio tools and SDKs are an exception - they may be installed in a directory which contains spaces. It is strongly recommended that you accept the default settings for all installation locations.

Quotation marks in `PATH`

Quotation marks (") aren't translated properly when passed to MozillaBuild sub-shells. Since they're not usually necessary, you should ensure they're not in your `PATH` environment variable.

Python failed to find files in directories

Python can sometimes fail to find files in directories when path length limits are reached, even when the root directory is kept relatively short:

`C:\mozilla-source\firefox`. This can be resolved by [turning Windows long paths on](#).

`PYTHON` environment variable

If `PYTHON` is set, the build may fail with the error: "`The system cannot find the file specified`." Ensure that you aren't having a `PYTHON` environment variable set.

Cygwin interference

If you happen to have Cygwin installed, its tools may erroneously be used when building Firefox. Ensure that MozillaBuild directories (in `C:\mozilla-build\`) are before Cygwin directories in the `PATH` environment variable.

